

Data Creators › Day 4 — run sheet

Day 4 run sheet — Bitemporal loading and Airflow orchestration

Instructor-only companion to the anchor notebook, in the notebook's order: the same numbered sections and labelled parts, each with its time, purpose, what is on screen, what to say, what to ask at the whiteboard (with the answers rooms give and what to add), the demo steps with their coaching, fallbacks, the transition and what to do if ahead or behind. Keep the notebook on the projector and this beside it. Times follow the instructor plan; adapt them to the room and keep the sequence.

Before the day

- Both demos build on Day 3's price sheet. Section 1 runs in `demos/price_sheet/` (its first cell resets the folder to Day 3's finished project). Section 2 runs the same project as the `price_sheet_daily` DAG on the local Airflow stack in `demos/airflow_local/`.
- Before the day: Docker Desktop running, then `bash demos/airflow_local/run.sh start` and `bash demos/airflow_local/run.sh seed`. The Airflow UI is http://localhost:8080. Run sections 1 and 2 once before the room arrives.
- Backups, not shown in the notebook: `bash demos/bitemporal/demo.sh` (the associate example) and the `portfolio_analytics_daily` DAG in the same stack (the learners' challenge DAG).

© 2026 Decoded Limited · decoded.com

Sections

#	Section	Time	What happens
1	Bitemporal loading	09:00–09:45	Demo: two clocks on the price sheet, three deliveries, three questions · Guest or volunteer: a real table · Shareback: which clock?
2	Airflow orchestration	09:45–13:30	Orientation: why orchestrate, and reading a DAG · Break · Demo: the challenge's steps 3 to 10 on the price sheet · Launch · Challenge 10: Airflow practical · Walkthrough: Phase 2 on the price sheet · Lunch · Shareback: one run, one log, one output check
3	CI/CD and release operations	13:30–15:00	Demo: GitOps, Harness, Datadog · Break · Activity 10: release decision · Activity 11: explain a failure and repair · Friday preparation

Timeline

§	Time	Block	Type	On screen
1	09:00–09:45	Bitemporal loading, on yesterday's price sheet	Demo + shareback	Anchor › section 1, top to bottom (demos/price_sheet); a guest or volunteer's screen

© 2026 Decoded Limited · decoded.com

§	Time	Block	Type	On screen
2	09:45–10:00	Orchestration and capstone orientation	Demo	Anchor › section 2, steps 1 and 2 (demos/airflow_local); Friday preview
2	10:00–10:15	Break	Break	
2	10:15–10:45	Airflow demonstration	Demo	Anchor › section 2, steps 3 to 10; the Airflow UI at localhost:8080
2	10:45–12:00	Challenge 10 — Airflow practical	Challenge	Anchor › Challenge box; learners in challenges/v1/03-airflow-orchestration
2	12:00–13:00	Lunch	Break	
2	13:00–13:30	Continue Airflow practical, regroup at 13:30	Challenge + shareback	Anchor › Challenge box
3	13:30–14:00	CI/CD and deployment context	Demo	Anchor › CI/CD (diagram), Harness walkthrough, Datadog monitors
3	14:00–14:15	Break	Break	
3	14:15–14:35	Release decision in pairs	Challenge + shareback	Anchor › Activity 10 card
3	14:35–14:50	Explain a failure and repair	Challenge + shareback	Anchor › Activity 11 card and template
3	14:50–15:00	Friday preparation and closing questions	Shareback	Anchor › Friday checklist

© 2026 Decoded Limited · decoded.com

1. Bitemporal loading

Demo — two clocks on the price sheet: three deliveries, three questions · 09:00–09:45

Purpose. Business time and system time; loading strategies; a worked history with a correction; which clock answers which question.

On screen. Anchor › section 1, top to bottom (demos/price_sheet); a guest or volunteer's screen

Instructor notes.

- Open with five pictures, one question each, in place of the whiteboard list: ways to load, what the source sends, the two clocks, a correction, UTC. Type the room's answers into the cell under each picture; each picture's answer is in this run sheet, not in the notebook. About ten minutes.
- Then eight steps on Day 3's price sheet, one command per cell: reset and build, look at the three FX deliveries, write the history model, write the macro, three loads with the GBP rows and a picture after each, then the three questions.
- The story: the vendor sends a wrong GBP rate for 4 November (1.628112, two digits swapped) and corrects it in the next delivery (1.268112). After load 3 GBP has four rows; name them A, B, C, D as the pictures do.
- The last query shows one GBP price in dollars twice, as was and as is: the link back to yesterday's price sheet, which could only ever show one.
- A volunteer or guest shows a real table for ten minutes if one was recruited yesterday; the demo covers the same ground either way.
- Shareback: each table answers one 'which clock?' question.
- Backup: bash demos/bitemporal/demo.sh runs the older associate example from one script.

Say.

- "The vendor sends a wrong rate on Wednesday and corrects it on Thursday. What was the rate, and what did Wednesday's report say?"
- "Yesterday's snapshot kept one clock. Two clocks tell a new day from a correction."

Picture: Four ways for new data to land in a table, and what each one remembers

- Ask: How can new data land in a table, and what does each way remember afterwards?
- Answer: Truncate and reload, and insert overwrite, keep only the latest. SCD type 2 adds a start and an end date, so it remembers when each value was true. Bitemporal is SCD type 2 with a second clock: it also remembers when the warehouse believed it.

© 2026 Decoded Limited · decoded.com

- Today: Yesterday’s price sheet reloads: it remembers nothing. This morning it gets a table that does.

Picture: What the source sends: everything, only the changes, or balances

- Ask: What does your source send each time: everything, or only what changed?
- Answer: Full: the whole book of record every run, so every row is compared and it is costly on a large table. Incremental: only new or changed records with a business key; a known key with new content ends the previous version. Balance: a strategy of its own for monetary balances.
- Today: The FX deliveries are incremental: each one carries the rates for one day, keyed by currency.

Picture: Two clocks on every row: when it was true, and when we believed it

- Ask: What are the two clocks, and where does each one get its time from?
- Answer: Business time is when the fact was true in the world: it comes from the data, here the day the rate is for. System time is when the warehouse loaded or believed it: it comes from the load. Each has a start and an end column, and the row also carries a current indicator and a row hash.
- Today: Yesterday’s snapshot kept one clock. Two clocks tell a new day from a correction.

Picture: A correction, overwritten or kept

- Ask: The vendor corrects Wednesday’s rate on Thursday. If we overwrite it, which question can nobody answer any more?
- Answer: What Wednesday’s report used. ‘As was’ must still give the wrong rate the report was built on, and ‘as of’ must give the corrected rate today. Overwriting destroys the first; a status flag cannot express either.
- Today: Load 3 in the demo is exactly this correction, and the last query shows both answers.

Picture: One run, two dates: the business date from the data, the system time from the load, in UTC

- Ask: A run at 03:00 Pacific on Thursday loads Wednesday’s rates. Which date goes on which clock?
- Answer: Wednesday on the business clock, because that is the day the rates are for. Thursday 11:00 UTC on the system clock, because the platform stamps system time in UTC. Both are right; always say which clock a date column is.
- Today: The history model stamps its system clock in UTC for the same reason.

The tables: from one clock to two

- Walk the tables in order and ask before each one: what would this table look like after Thursday’s correction?
- Table 1: one row per currency per day. A correction replaces the value, and nothing in the table says so.

© 2026 Decoded Limited · decoded.com

- Table 2: SCD stands for slowly changing dimension. Type 2 means: when a value changes, add a new row and close the old one. Each row has one pair of dates saying when that version was valid. It can answer: what was the rate on 3 November? Yesterday's dbt snapshot is SCD type 2: `dbt_valid_from` and `dbt_valid_to` are that one pair of dates.
- Table 3: on Thursday the vendor says Wednesday's rate was wrong. With one pair of dates there are two choices, and each loses something. Let the room argue for each choice before showing what it loses. One clock cannot tell "the world changed" from "we found out we were wrong".
- Table 4: the business clock (business from, business to) is when the value was true in the world. The system clock (system from, system to) is when the warehouse believed it. A correction closes the wrong row's system clock and adds the right value for the same business date. Nothing is overwritten, and nothing false is recorded.
- Table 5: with Tuesday's rate included, GBP has four rows. Rows A and B hold the same rate: A is what we believed until Wednesday's load; B says it is still true, for 3 November only.
- Ask: which row answers "what did Wednesday evening's price sheet use for 4 November?" Row C.
- Table 6: SCD type 2 is simpler; bitemporal queries must say which clock they mean. "What did we believe at time T?" is what a past report said.
- Table 7: the business clock is the history of the world. The system clock is the history of what we knew. A correction can arrive a day or a year later: it is about a past date, however late it comes. Together they answer the question an auditor asks: as we knew it at time T, what was true on date D?
- SCD type 2 is enough when data changes but is rarely corrected, and nobody has to reproduce a past report. Bitemporal is for audit and regulated reporting.
- One line to leave them with: SCD type 2 remembers what was true; bitemporal also remembers what we thought was true.

The tables: one load, before and after

- A bitemporal load only ever does two things: it closes a system clock, and it adds rows.
- Here it closes row C (two cells change) and adds row D. No rate is ever edited.
- This is load 3 in the demo. The times in the demo are the seconds the cells ran, not Wednesday and Thursday.

On the commands.

- `%cd {HERE}/demos/price_sheet` Day 4's copy of yesterday's price sheet. dbt commands run from the project folder, as yesterday.
- `bash reset.sh` The starting point: Day 3's finished project. Four models, the loop model, the to_usd macro and the tests.
- `dbt seed` The raw data first, as yesterday. One new seed today: fx_deliveries.
- `dbt build --exclude resource_type:seed` The models and the tests, in graph order: yesterday's price sheet, built and passing.
- `dbt show --inline "select delivery_id, from_ccy, rate_dt as rate_for,…` Three deliveries in one seed. D1: the rates for 3 November. D2: the rates for 4 November, with a wrong GBP rate (1.628112: two digits swapped). D2-CORR: the vendor sends 4 November again, corrected (1.268112).

© 2026 Decoded Limited · decoded.com

Picture: What the macro does with each row of a new delivery

- The model names a strategy dbt does not know, `bitemporal_load`. dbt looks for a macro called `get_incremental_bitemporal_load_sql` and uses it: found in `macros/`, as yesterday.
- A different rate with a later business date is a new day. A different rate for the same business date is a correction.
- Rows are only added. The one update is closing a system clock, so nothing is lost.
- CG's platform does the same job with `bitemporal_load_v2`, built on Delta `MERGE`.

On the commands.

- `dbt run --select fx_rates_history --vars "delivery: D1"` –vars passes a value into the model: which delivery to load. The table does not exist yet, so dbt creates it from the SELECT; the macro is not needed on the first run.
- `dbt show --inline "select cast(fx_rate as varchar) as fx_rate, dl_bus…` One row for GBP: the rate for 3 November, believed since load 1, open on both clocks.
- `dbt run --select fx_rates_history --vars "delivery: D2"` The table exists now, so the macro runs: close, carry forward, insert.
- `dbt show --inline "select cast(fx_rate as varchar) as fx_rate, dl_bus…` Three rows. Row A: its system clock is closed. Row B: the same rate, carried forward with its business clock ended at 3 November. Row C: the new rate for 4 November, which is wrong, and current.
- `dbt run --select fx_rates_history --vars "delivery: D2-CORR"` The same business date, a different rate: a correction.
- `dbt show --inline "select cast(fx_rate as varchar) as fx_rate, dl_bus…` Four rows. Row C, the wrong rate, has its system clock closed: it is kept, not erased. Row D is the corrected rate for the same day. Nothing was carried forward this time: a correction replaces a version, it does not end it.
- `dbt show --inline "select from_ccy, cast(fx_rate as varchar) as fx_ra…` AS IS: what is true now. Both clocks open.
- `dbt show --inline "select from_ccy, cast(fx_rate as varchar) as fx_ra…` AS OF 3 November, with today's knowledge: the business clock covers the date, the system clock is open.
- `dbt show --inline "select from_ccy, cast(fx_rate as varchar) as fx_ra…` AS WAS, just before the correction arrived: the system clock covers that moment. GBP shows the wrong rate, because that is what we believed then.
- `dbt show --inline "select p.instrument_id, p.price_date, cast(p.price…` The same price in dollars, twice: what Wednesday's price sheet said, and what it says now. Both answers come from one table.

© 2026 Decoded Limited · decoded.com

Guest or volunteer — a real bitemporal table · 09:00–09:45

Shareback — which clock? · 09:00–09:45

Transition. From history to timing: orchestration.

If ahead or behind. Over time: of the five opening pictures ask only the two clocks and the correction; show the macro picture but not the macro file; keep the three loads and the three questions.

© 2026 Decoded Limited · decoded.com

2. Airflow orchestration

Orientation — why orchestrate, and reading a DAG · 09:45–10:00

Purpose. What orchestration is, on yesterday's price sheet; how to read a DAG file; a preview of Friday's work and its starting files.

On screen. Anchor › section 2, steps 1 and 2 (demos/airflow_local); Friday preview

Instructor notes.

- Steps 1 and 2 of the challenge on the price_sheet_daily DAG: why orchestrate (the five-gaps picture), the path, the DAG picture, then the top of the DAG file and the DAG block.
- The first cell puts the TODO back (run.sh unwire), so the DAG starts as the learners' does.
- Friday: Phase 1 scope, two to three hours, the working folder rule.

Say.

- "Airflow plays no instrument. It tells dbt when to play, and what to do when it goes wrong."

Picture: The DAG we will run: a gate, yesterday's models as tasks, a publish step

- The freshness gate runs first, so a stale feed stops the run before anything is built.
- Cosmos turns the dbt project into one task per model, then the tests, in dbt's own order.
- `publish_to_catalog` is the deliverable: it carries the 90-minute SLA.
- `start` and `end` do no work: they make the graph easy to read.

On the commands.

- `%cd {HERE}/demos/airflow_local` The local Airflow stack. From here on, the commands are the ones learners type in their container.
- `bash run.sh unwire` The starting point: the DAG as learners get it. Every task is built; the one line that connects them is a TODO.
- `sed -n '1,16p' dags/price_sheet/dags/price_sheet_daily.py` Read the top of the file. What runs first, and why? Which task carries the SLA? Where does dbt run?

© 2026 Decoded Limited · decoded.com

- `sed -n '/^default_args/,/^) as dag:/p' dags/price_sheet/dags/price_sh…` default_args: what every task inherits. Then the DAG itself: its id (the same as the file name), its schedule, its start date, its tags.
- `python -c "import ast; ast.parse(open('dags/price_sheet/dags/price_sh…` The cheapest check: is the file valid Python? No Airflow needed.
- `astro dev run dags list-import-errors | grep -v "secrets backend"` The real check: did Airflow load every DAG file? “No data found” is the good answer. Airflow runs inside containers, so its commands start with: astro dev run

Say.

- “Friday: you get CSV files about a fictional wealth manager called Atlas and a README. Phase 1 is the certification bar: sources, the three layers, tests, and understanding the orchestration. Two to three hours for most teams. Phase 2 is fourteen extensions you choose from. We watch you build it; nobody hands out solutions.”
- “Work in data_products/atlas_elt, not in the challenge folder. If you run it in the wrong folder, that is on you, and now you have been told.”

Transition. Break, then the Airflow demo.

> ☕ Break · 15 min · back at 10:15

Demo — connect, gate, group, wire, schedule, retry, run, repair · 10:15–10:45

Purpose. Steps 3 to 10 of the challenge on the price sheet: the connection, the freshness gate, the Cosmos group, the one line of wiring, the schedule, retries, a green run, and a controlled failure with its repair.

On screen. Anchor › section 2, steps 3 to 10; the Airflow UI at localhost:8080

Instructor notes.

- Start the stack before the day (bash run.sh start). Every step is one command per cell; keep the UI open beside the notebook and switch to it at steps 6, 9 and 10.
- Step 6 is the learners’ one line: run.sh wire replaces the TODO. Read >> as ‘and then’.
- Step 9: a green run takes about half a minute here. The dbt show cell is the evidence, not the green boxes.
- Step 10: stale, trigger, wait about a minute: the gate goes yellow twice, then red, and the rest is upstream_failed. Then fresh, and Clear the failed task in the UI with Include Downstream. The demo DAG waits 15 seconds between retries; the learners’ waits 2 minutes, so tell them a failure takes about ten minutes to turn red.
- Close with the blueprint picture: at CG most DAGs come from a blueprint and YAML.

© 2026 Decoded Limited · decoded.com

- Backup: portfolio_analytics_daily, the learners' own DAG, is in the same stack (bash run.sh trigger, wait, status with no DAG name).

Say.

- "A green graph is not evidence that the data is right. The rows are the evidence."
- "You write one line today. Everything else, you read and prove."

Picture: Airflow's components, locally and hosted

- The DAG processor parses dags/ and reports import errors; the scheduler decides what is due; workers run tasks; the triggerer handles deferred waits; the API server serves the UI.
- astro dev start runs all of them as containers on the laptop; a hosted Astro deployment is the same thing run for you and vended per team.

On the commands.

- `bash run.sh dbt price_sheet debug --connection | tail -3` dbt debug, run inside the scheduler container, in the price sheet project. Learners: (cd data_products/portfolio_analytics/dbt && dbt debug), and the answer comes from Databricks.
- `bash run.sh seed 2>&1 | grep -E 'Done|rror'` The vendor delivers: the raw feeds land in the warehouse. Learners do this once in the prerequisites: dbt seed.

Picture: The freshness gate: how old is the feed, and does the run carry on?

- `dbt source freshness` reads the newest `vendor_ts` and compares it with now.
- Only an `error_after` breach fails the command. A `warn_after` breach prints a warning and the run carries on, so where you put `error_after` is the real decision.
- A `PythonOperator`, not a `BashOperator`: CG governance blocks operators that run arbitrary shell, so the dbt call is wrapped in a Python function.

On the commands.

- `grep -n -B1 -A4 'loaded_at_field' dags/price_sheet/dbt/models/sources…` The contract for the feed: which column says when it arrived, when to warn, when to stop.
- `sed -n '/^def _check_source_freshness/,/^ )/p' dags/price_shee…` The task's function: run dbt source freshness, and raise an error if it fails. Raising is what fails the task.
- `astro dev run tasks test price_sheet_daily source_freshness_check 202…` Run this one task on its own: no schedule, no waiting for the tasks before it. PASS, then new_state=success.

Picture: Cosmos reads the dbt project and makes one Airflow task per model, in dbt's order

© 2026 Decoded Limited · decoded.com

- The dbt project already knows its own graph, from `source()` and `ref()`. Cosmos reuses it.
- It needs to know where the project is (`ProjectConfig`) and how to connect (`ProfileConfig`, which reads the project's `profiles.yml`).
- `TestBehavior.AFTER_ALL`: the tests run once, after every model is built.

On the commands.

- `sed -n '/^ dbt_pipeline = DbtTaskGroup/,/^ )/p' dags/price_shee…` The whole dbt project in one call. Nobody writes a task per model.
- `astro dev run tasks list price_sheet_daily | grep -v "secrets backend"` Five tasks named dbt_pipeline._run and one dbt_test: yesterday's models. Plus the four tasks around them.
- `grep -n "TODO" dags/price_sheet/dags/price_sheet_daily.py || echo "no…` Five tasks exist, and nothing connects them yet.
- `bash run.sh wire` The TODO replaced by one line. Read >> as "and then". The task group counts as one step: it waits for the gate, and publish waits for all of it.
- `astro dev run dags reserialize > /dev/null 2>&1; echo 'Airflow has re…` Ask Airflow to read the DAG files now, not at its next scan.
- `python -m pytest dags/price_sheet/tests -q -k chain` The check that the chain is wired. Learners: mise run challenge:verify 03. Then open the Graph view in the UI: five boxes, left to right.

Picture: Reading a cron schedule: five fields, four symbols

- Read the five fields left to right: minute, hour, day of month, month, day of week. Smallest unit first.
- A star means every value. A comma is a list, a dash is a range, a slash is a step.
- Day of week runs 0 to 6 with Sunday as 0, so 1-5 is Monday to Friday. Names work too: MON-FRI.
- Ask the room to read each example aloud before showing the answer. crontab.guru checks any expression.
- Airflow also accepts presets: `@hourly`, `@daily` (midnight), `@weekly`; and `None` for a DAG that only runs when triggered.

Picture: The schedule: five cron fields, and a start date that carries the time zone

- `0 6 * * 1-5` is 06:00, Monday to Friday. Weekdays only: markets are closed at the weekend.
- Airflow stores times in UTC. A start date with no time zone would let 06:00 drift by an hour when the clocks change; `Europe/London` pins it.
- `max_active_tasks` is a third setting: how many tasks run at once inside one run. Here 1, because the demo warehouse takes one writer; the learners' DAG has 4.

On the commands.

- `astro dev run dags details price_sheet_daily 2>&1 | grep -i -E "timet…` The schedule as Airflow understood it.

Picture: Retries: a task that fails waits and tries again, and only alerts a person when the retries run out

© 2026 Decoded Limited · decoded.com

- `retries: 2` means three attempts. `retry_exponential_backoff` doubles the wait each time.
- A task waiting to retry is `up_for_retry`, yellow in the UI: the policy is working, the task is not stuck.
- The demo waits 15 seconds, so you see it within a minute. The learners' DAG waits 2 minutes, then 4: a failure takes about ten minutes to turn red.

On the commands.

- `sed -n '/^default_args/,/^}/p' dags/price_sheet/dags/price_sheet_dail…` Set once, inherited by every task.
- `astro dev run dags unpause price_sheet_daily | grep -v "secrets backe…` A new DAG starts paused: Airflow knows it and will not run it. "No paused DAGs were found" means it was unpaused before.
- `astro dev run dags trigger price_sheet_daily 2>&1 | grep -c queued` Ask for one run now, on top of the schedule. 1 means one run was queued.
- `bash run.sh wait price_sheet_daily` Watch it in the UI while this polls: Grid (runs over time), Graph (the tasks of one run), then a task's Logs.
- `bash run.sh status price_sheet_daily` The run, then the state of every task in it. Learners: astro dev run dags list-runs, then tasks states-for-dag-run.
- `bash run.sh dbt price_sheet show --select fct_prices_usd --limit 5` The proof is in the warehouse, not in the green boxes: the same rows a hand-typed dbt build produces.

Picture: A failed run: read the log, fix the cause, then clear the failed task to re-run it and everything after it

- Clearing re-queues the task and the tasks after it, in the same run. No new run is created.
- Clear from the UI: it clears the one run you clicked. The command-line version selects by date and can clear several runs at once.
- 'Mark success' skips the check instead of fixing it: the tasks after it run on whatever data is there.

On the commands.

- `bash run.sh stale price_sheet_daily` Break it on purpose: the vendor feed now looks two days old.
- `bash run.sh wait price_sheet_daily` About a minute. In the UI the gate goes yellow (up_for_retry) twice, then red.
- `bash run.sh status price_sheet_daily | grep -E 'task_id|freshness|dbt…` The gate failed; everything after it is upstream_failed. Nothing was built and nothing was published: yesterday's price sheet stands. In the UI: click the red task, then Logs, and read the first useful line.
- `bash run.sh fresh price_sheet_daily` Fix the cause: the feed arrives. Then, in the UI: click the failed task, Clear, tick Include Downstream, Confirm.
- `bash run.sh wait price_sheet_daily` After the Clear in the UI: the same run, green this time. (No browser? Trigger a new run with the cell above.)

Picture: Astro blueprints: YAML in, a DAG out

- A blueprint is Python written once by the platform team with placeholders; a dataset's YAML fills them; the result is a registered DAG that triggers a Databricks job.

© 2026 Decoded Limited · decoded.com

- Over ninety percent of products fit a blueprint; a custom DAG is for the rest, and can become a new blueprint.
- The challenge is Python on purpose: once the Python makes sense, the YAML is just configuration.

On the commands.

- `sed -n '1,30p' ../../../reference/DataCreators-client/dev-containe…` A platform blueprint from the vended repository: the Python learners never write, filled from YAML.

Launch — from this demo to your challenge · 10:15–10:45

Picture: From this demo to the challenge: the same ten steps, on the Portfolio Analytics project

- Your DAG is `portfolio_analytics_daily`: the same five boxes, with yesterday's Portfolio Analytics models inside the group.
- It runs on Databricks, so a run takes minutes, and a failed task takes about ten minutes to turn red.
- Phase 1 has one line to write. Every other section asks you to read a piece and prove it works.

Transition. Challenge 10 until lunch.

If ahead or behind. Short of time: steps 4, 6, 9 and 10. If the stack misbehaves, the status cell still shows the earlier runs.

Challenge 10 — Airflow practical · 10:45–12:00

Purpose. Launch the assigned Airflow project; learners wire the chain, run tasks inside the scheduler, inspect states and logs.

On screen. Anchor › Challenge box; learners in challenges/v1/03-airflow-orchestration

Instructor notes.

- Boot order matters: setup, then seeds, then astro dev start; the cold-start symptom and fix are in the anchor.
- Collect deployment and monitoring questions for the afternoon.

Say.

- "Which run produced this result, and what proves the data is right?"

Challenge — Airflow practical (challenge 03). `challenges/v1/03-airflow-orchestration/README.md` · until 13:30, with lunch.

© 2026 Decoded Limited · decoded.com

Walkthrough — Phase 2: production operations, on the price sheet · 10:45–12:00

Demo — Phase 2 on the price sheet. The ten Phase 2 challenges, one or two cells each, for when learners reach them.

1. Use these cells at 13:00, or beside a learner who has reached Phase 2.
2. Every Phase 2 file ships complete: the task is to read each piece and prove it does what it says.
3. The failed run from step 10 is the evidence for challenges 1 to 4.

On the commands.

- `sed -n '/^def send_alert/,$p' dags/price_sheet/dags/notifications.py` Every chat message goes through send_alert. The webhook address lives in an Airflow connection called alert_webhook, not in the code.

Picture: Four events, four callbacks, and who hears about each

- A page wakes a person, so it is kept for a task that failed after all its retries: the price sheet will be late.
- An SLA miss means slow, not down: a message, no page. A retry is expected: a ticket, so a flaky task leaves a trail.
- No Slack and no PagerDuty at CG: Datadog is the automatic sink, support pages through xMatters, Teams is the chat.

On the commands.

- `grep -n '^def \|P[123]' dags/price_sheet/dags/callbacks.py | grep 'de…` Each function, then the tier it serves: the two transports (Datadog, ticket) and the four callbacks.
- `bash run.sh alerts price_sheet_daily` What the callbacks did in the failed run from step 10: a ticket for each retry (P3), then a Datadog event (P1) and a Teams message (P2). No keys are set on this laptop, so each one logs instead of sending.
- `grep -n "_callback\|sla=" dags/price_sheet/dags/price_sheet_daily.py …` Three places: default_args (every task), the DAG (the SLA miss), and the last working task (success once per run, and the SLA itself).
- `bash run.sh dbt price_sheet source freshness --select source:vendor_f…` The same check the gate runs. warn_after 4 hours, error_after 6 hours: a feed five hours old only warns, and the run goes ahead. Is six hours the right line for a 07:30 deadline? That is a decision to defend, not a default.

Picture: Alerting says something broke; monitoring shows something drifting

- Five panels cover most pipelines: run status, duration, task failures, SLA compliance, source freshness.
- The sixth catches a feed that arrives on time but short. Freshness watches the clock; volume watches the row count.

© 2026 Decoded Limited · decoded.com

On the commands.

- `grep -E 'name:|alert_condition:|alert_channel:' dags/price_sheet/incl…` Each panel: a metric, when it alerts, and where the alert goes.
- `grep -n '^## \|^### Alert' dags/price_sheet/include/runbooks/price_sh…` The runbook: what the on-call engineer opens when an alert lands. Five alerts, each with symptom, diagnosis and remediation.
- `sed -n '1,16p' dags/price_sheet/include/launch/consumer-guide.md` The consumer guide: what a new user of the price sheet reads. Both documents live beside the DAG, so they change in the same pull request.
- `python -m pytest dags/price_sheet/tests -v --no-header 2>&1 | grep -E…` No Airflow and no warehouse: the files parse, the chain is wired, the callbacks and the SLA are attached, and no file holds a secret.

Picture: Two ways to run dbt from Airflow: on the worker (Cosmos), or inside Databricks (CG's paved path)

- Cosmos: one Airflow task per model, so you see which model failed.
- Submit-run: each Airflow task submits a Databricks job and waits. The job is one box; its logs live in Databricks.
- The challenge's second DAG wires the failure and SLA callbacks but not the retry and success ones. Spotting that gap is the exercise.

On the commands.

- `sed -n '1,16p' ../../../reference/DataCreators-client/dev-containe…` The challenge's second DAG: the same pipeline, as jobs submitted to Databricks.

Transition. Lunch.

If ahead or behind. Fast finishers: the extension questions (late input, failed check, repeated run), then Phase 2.

> 🍽 Lunch · 60 min · back at 13:00

Shareback — one run, one log line, one output check per pair · 13:00–13:30

Purpose. Resolve blockers; Phase 2 for those on it; regroup with one run, one log and one output check per pair.

On screen. Anchor › Challenge box

Instructor notes.

- Keep credential and path errors separate from the intended data-quality failure.

© 2026 Decoded Limited · decoded.com

- Shareback: a pair shows the current run, a useful log line, and the warehouse output.
- For learners in Phase 2: the anchor's Phase 2 walkthrough shows each of the ten challenges on the price sheet, with the morning's failed run as the evidence for the alert tiers. Ten minutes on screen at 13:00 if several pairs are there.

Say.

- "Show me the run ID, the log line that told you what happened, and the table it produced."

Transition. CI/CD and the deployment picture.

© 2026 Decoded Limited · decoded.com

3. CI/CD and release operations

Demo — GitOps, Harness, Datadog · 13:30–14:00

Purpose. Where code goes, how deployment is initiated, where execution is visible, how failures are investigated: Harness, Astro, Datadog.

On screen. Anchor › CI/CD (diagram), Harness walkthrough, Datadog monitors

Instructor notes.

- Agree the agenda with a guest in advance; otherwise the anchor's walkthrough with screenshots.
- Three pictures, one question each, in place of the whiteboard list: what CI and CD mean, what it gives and costs, what Harness is. Say the costs too; rooms respect it.

Say.

- "A pull request is where acceptance happens. A deleted test is a block of red lines that is easy to skim past."

Picture: From a branch to production: pull request, merge, Harness, SQA, production with a change record

- Ask: What do CI and CD stand for, and where is each one in this picture?
- Answer: Integration is merging a feature branch, and continuous means daily or hourly, not monthly. Deployment is releasing to an environment, production included. A merge fires a webhook; Harness builds, tests and deploys. With GitOps the Git history becomes the history of what was live; CG is partway there: events trigger pipelines, with an approval step between.
- Today: This morning's DAG and yesterday's dbt project take this road to production.

Picture: What CI/CD gives, and what it costs

- Ask: What does CI/CD give a team, and what does it cost?
- Answer: It gives faster feedback that something is broken, a shared view of the product, and a record of the lifecycle. It costs too: automation with no real person in the loop, failures that arrive together when one shared system is down, and the maintenance, approval and audit work. It is not free.
- Today: A pull request is where acceptance happens. A deleted test is a block of red lines that is easy to skim past.

© 2026 Decoded Limited · decoded.com

Picture: Harness: the creator opens and merges a pull request; Harness does the rest

- Ask: What is Harness here, and how much of it does a creator touch?
- Answer: The CI/CD system, roughly GitOps: an approved pull request starts the pipeline. It keeps the deployment machinery away from the creator, who reads it more than operates it: which pipeline ran, where the logs are, who deployed last night. SQA is shared, so check who deployed last; production needs a change record.
- Today: The walkthrough below follows one deployment to SQA and then to production.

Demo — Harness, with screenshots or a guest. Walk a deployment to SQA and then to production: live if someone has access, else from the pre-flight screenshots.

1. Pick the environment and find the repository's pipeline.
2. SQA: Run, apply the SQA input set, choose the feature branch, run; a few minutes.
3. Check who deployed to SQA last and whether they finished testing: SQA is shared.
4. Production: the PRD pipeline, fixed values, the release branch, and a change record (CRQ) number before anything runs.
5. Open the logs of a failed execution.
6. Say: secrets live in AWS Secrets Manager, never in code or a chat; an API token that lets an AI assistant trigger deployments is a risk, not a shortcut.

On the commands.

- `grep -E 'name:|alert_condition:|alert_channel:' demos/airflow_local/d…` The price sheet's monitoring config again: six panels, each with an alert condition and a channel. This is what a Datadog monitor is for: the promises from Day 2, watched.

Transition. Break, then the release decision.

> ☕ Break · 15 min · back at 14:15

Activity 10 — release decision · 14:15–14:35

Purpose. A failed candidate with 500 rows and 497 distinct IDs, yesterday's accepted output available: decide what ships and what to tell the consumer.

On screen. Anchor › Activity 10 card

Instructor notes.

© 2026 Decoded Limited · decoded.com

- Ten minutes deciding and drafting, ten discussing. The failed candidate stays blocked; whether yesterday's output is acceptable depends on the consumer's freshness commitment.

Say.

- "What information is missing, and who can supply it?"

Activity 10 — Release decision. Pairs, 20 min; the card is in `activities/`.

Discussion notes for Activity 10

- The failed candidate stays blocked: three IDs appear twice, and until someone knows why, publishing it would publish a lie about uniqueness.
- Yesterday's accepted output is acceptable only if the consumer's freshness commitment allows a one-day-old sheet; the contract says, not the engineer.
- Missing information: which three IDs, whether the source re-delivered, whether the dedup rule or the source changed, who owns the source. The consumer update names what is known, what is not, and when the next update comes.
- Who overrides a failed gate, and where that decision is recorded, is a governance question worth leaving open in the room.

Transition. Write the failure note.

Activity 11 — explain a failure and repair · 14:35–14:50

Purpose. A short note another creator could use to locate the run and understand what happened; swapped and tested.

On screen. Anchor › Activity 11 card and template

Instructor notes.

- If a pair's execution was blocked, they use the anchor's prepared failure example and separate expected from observed.

Say.

- "Could another creator find this run from your note alone?"

Activity 11 — Explain a failure and repair. Pairs, then swap, 15 min; the card is in `activities/`.

Transition. Friday preparation.

© 2026 Decoded Limited · decoded.com

Friday preparation — the checklist, then the close · 14:50–15:00

Purpose. Check the starting files and environment; record remaining problems and who resolves them.

On screen. Anchor › Friday checklist

Instructor notes.

- Every learner opens the capstone README and runs the readiness checks; problems go on the list with a name.

Say.

- “Tomorrow starts sharp at nine. The sharper you start, the sharper you finish.”

Before learners leave

- [ ] The DAG, run and task identities, useful logs, an output check and a short explanation of the failure and repair, per pair.
- [ ] Every learner has opened challenges/v1/04-atlas-capstone/README.md and run the readiness checks in data_products/atlas_elt.
- [ ] A list of setup issues that could stop Friday’s start, each with the person responsible for the next action.
- [ ] The Airflow stack stopped on the learners’ laptops (astro dev stop) so Friday starts clean.

© 2026 Decoded Limited · decoded.com